



Implementation of Cryptographic Algorithms into VHDL

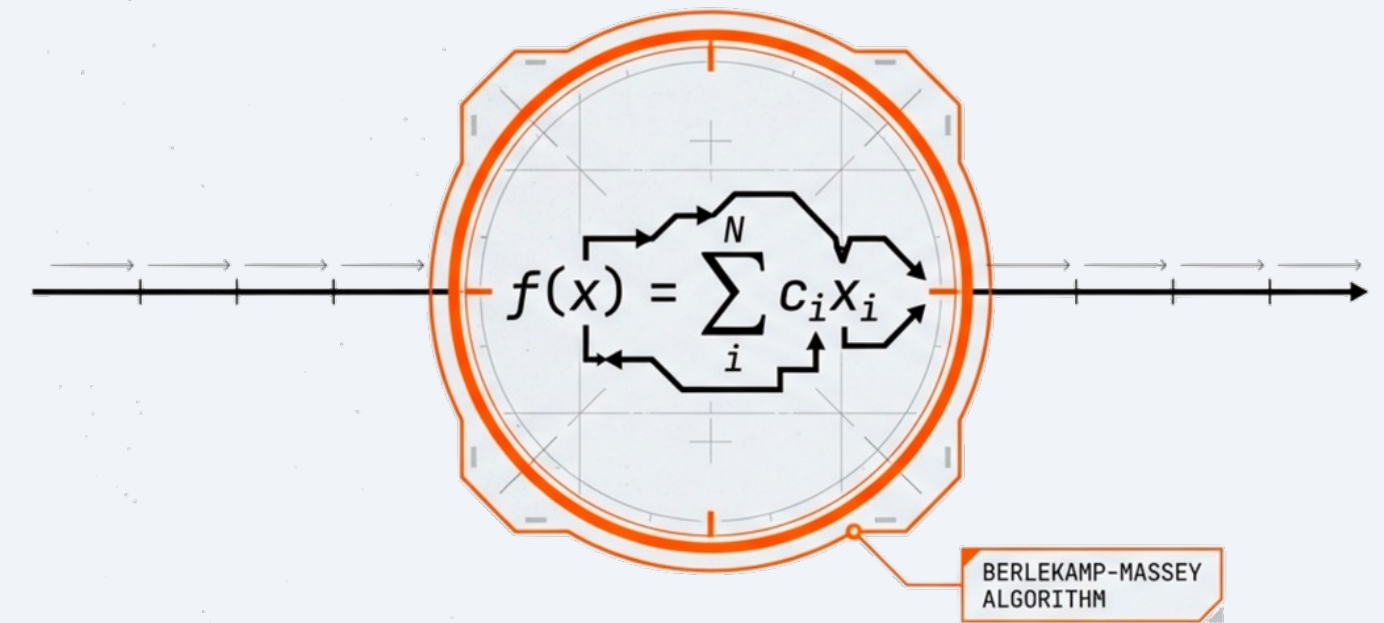
Orhun BOYDAK(21290703)

Ankara University | Department of Electrical and Electronics Engineering

Road Map

- Project 1: Production of Hard-to-Break Keys (LFSR Shrink Generator)
- Project 2: Hardware Accelerator for RSA (Modular Exponentiation)

LFSR Shrink Generator: Linear Weakness of LFSR



The Component

Linear Feedback Shift
Register (LFSR).
Fast and area-efficient.

The Flaw

Mathematically Linear.
Capturing 2N bits allows
reverse-engineering in
seconds.

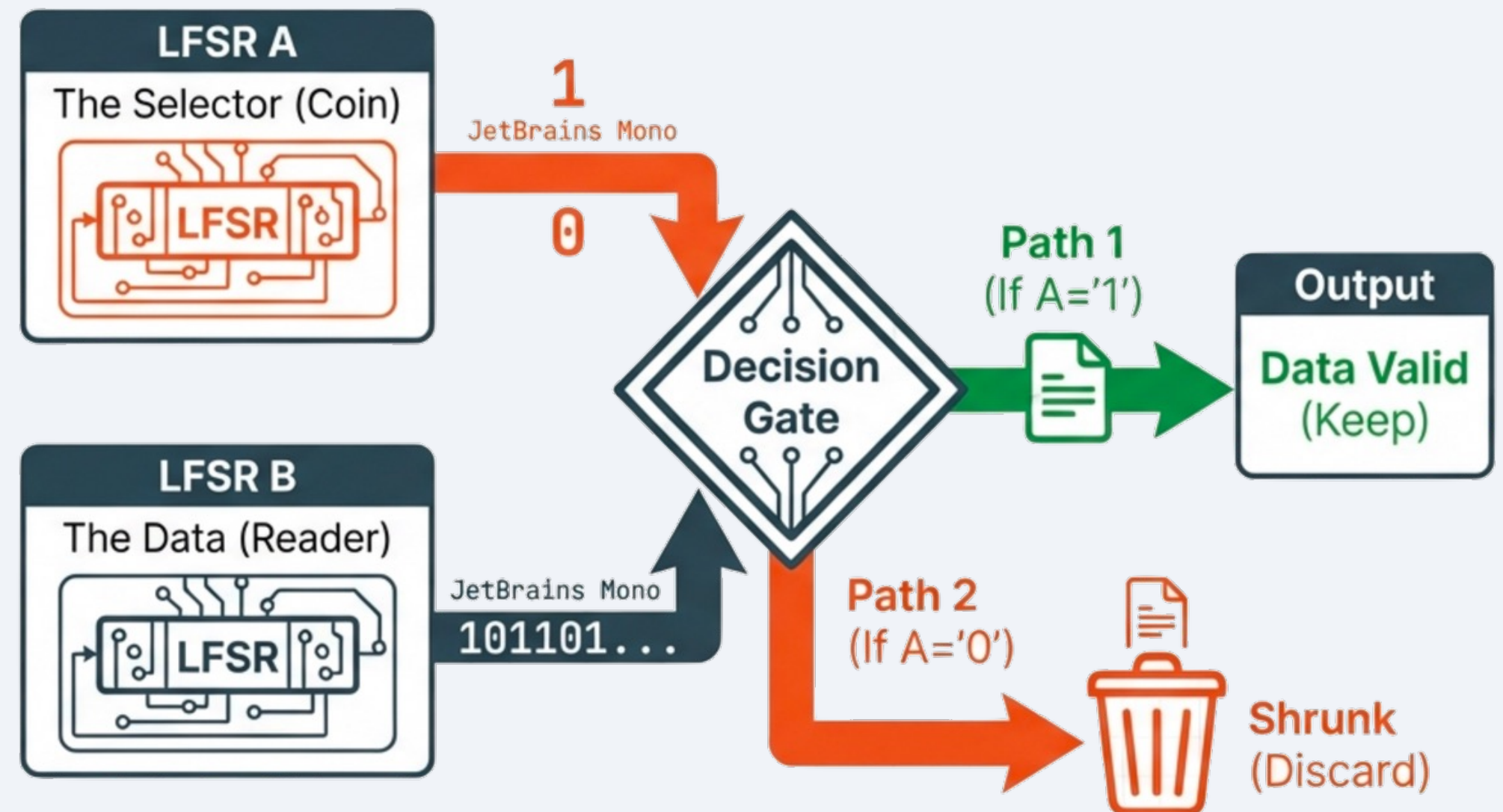
The Goal

Break the
linearity.

The Solution: Shrinking Generator Logic

Security Impact

An attacker sees a stream with missing bits. The time-correlation is destroyed. It is impossible to determine if bits were consecutive or if gaps exist.



Mathematical Robustness: Maximizing the Period

1. Design Choice

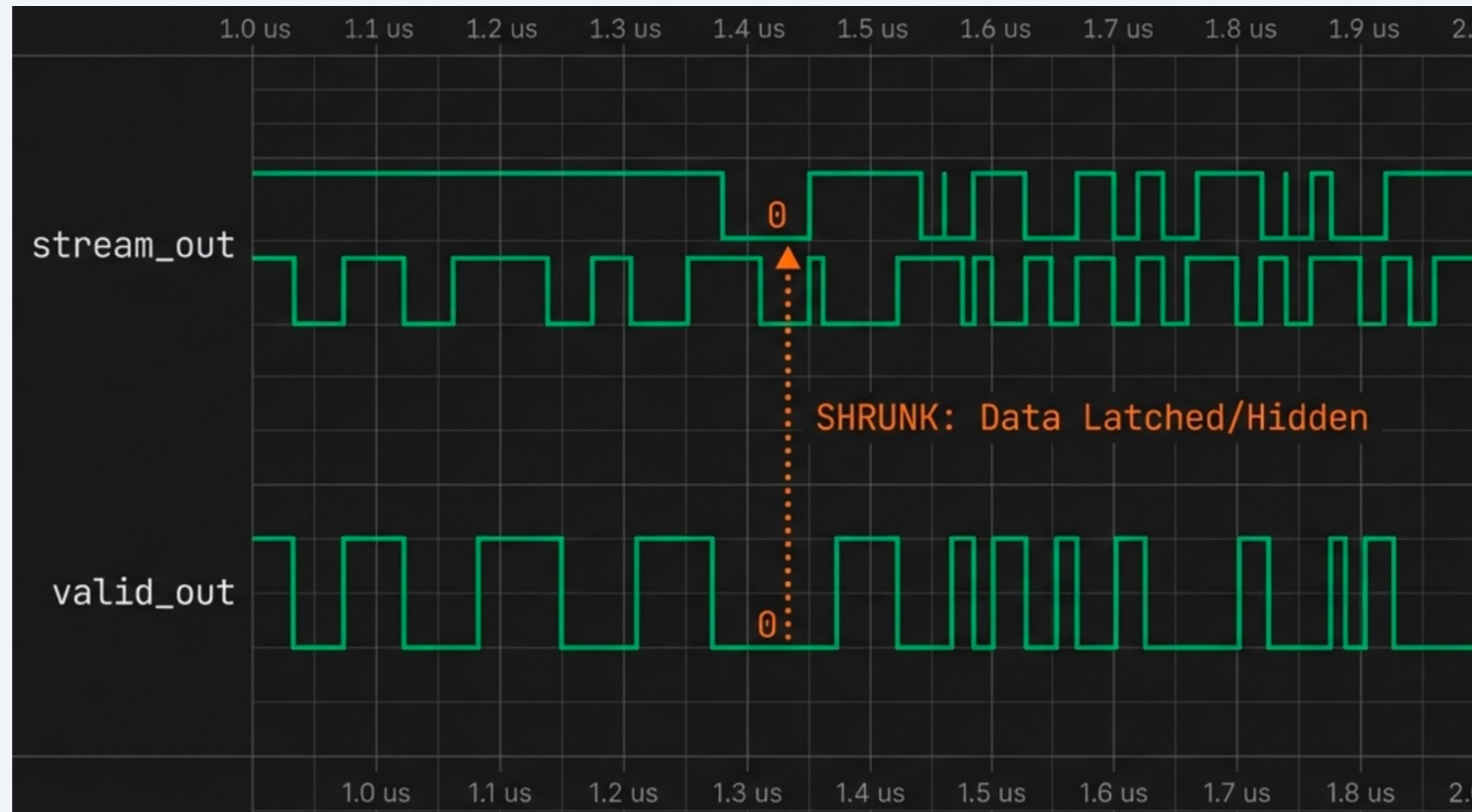
- Selector Length: 64-bit (2^6 , only divisible by 2)
- Generator Length: 63-bit ($3^2 * 7$, divisible by 3, 7)
- Result: $\text{GCD}(64, 63) = 1$. The sequences are mathematically independent.

2. Primitive Polynomials

- 64 Bit: $x^{64} + x^4 + x^3 + x + 1$
- 63 Bit: $x^{63} + x + 1$

The system period is maximized to
approx 2^{127} bits-longer than the age of the universe.

Evidence of Non-Linearity



The RSA Challenge: Modular Exponentiation

The Math

$$C = M^e \bmod N$$

- Square and Multiply: Scan bits.
If "0" → Square.
If "1" → Square and Multiply.

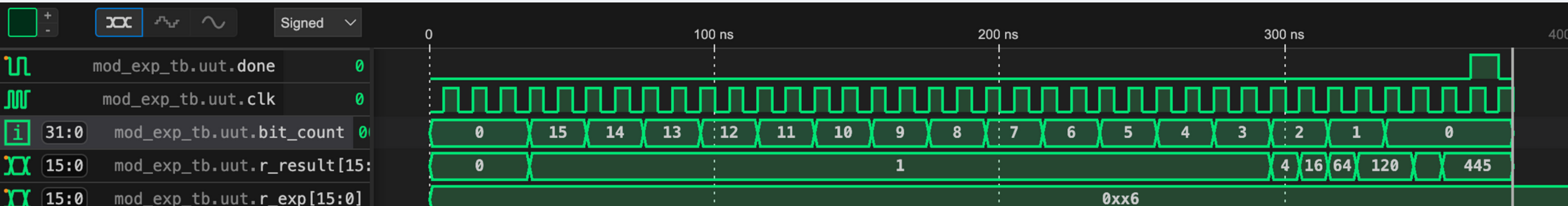
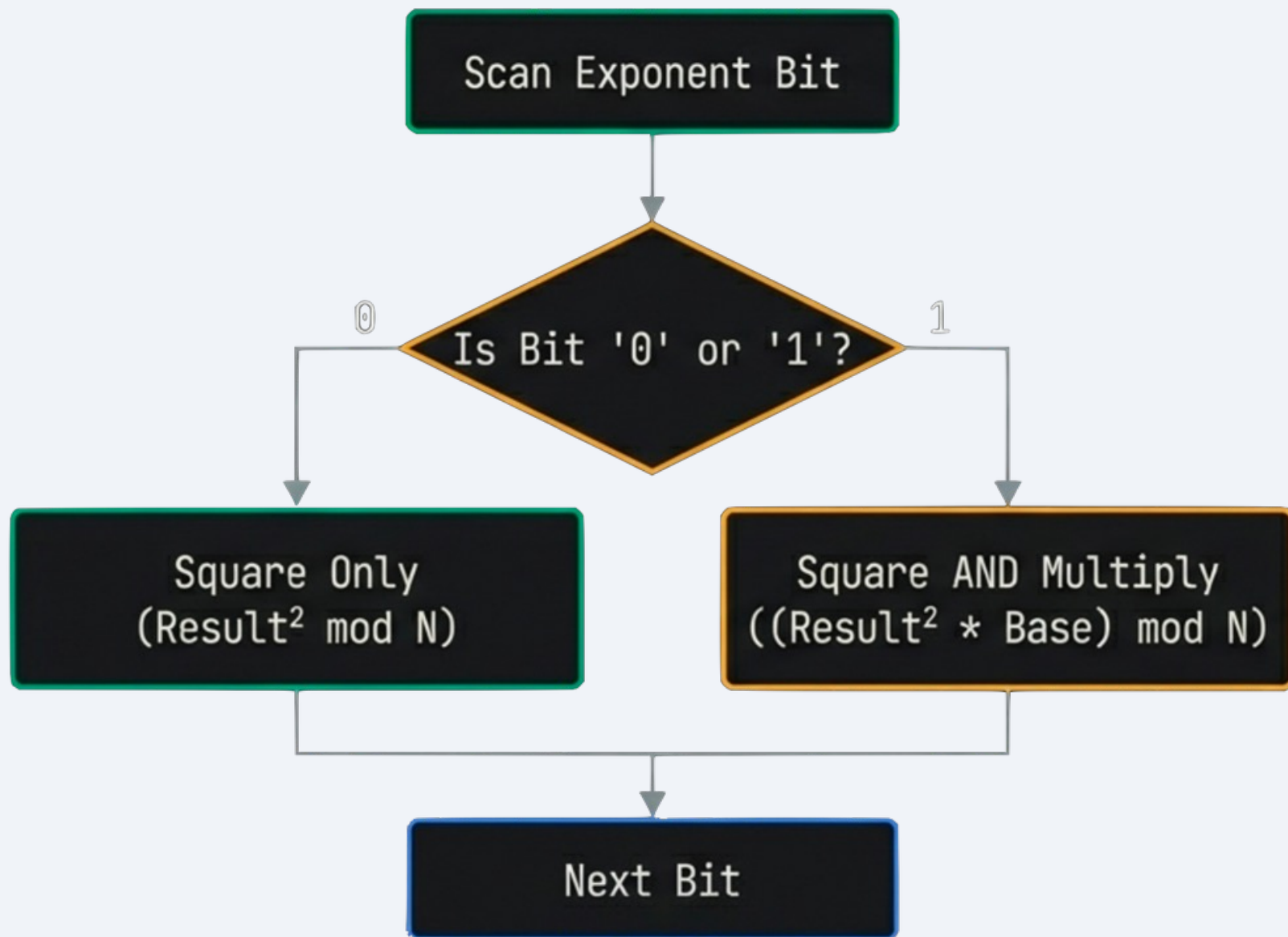
Hardware Shortage

- Division is computationally expensive.
- Consumes massive Silicon area.
- Creates high latency.

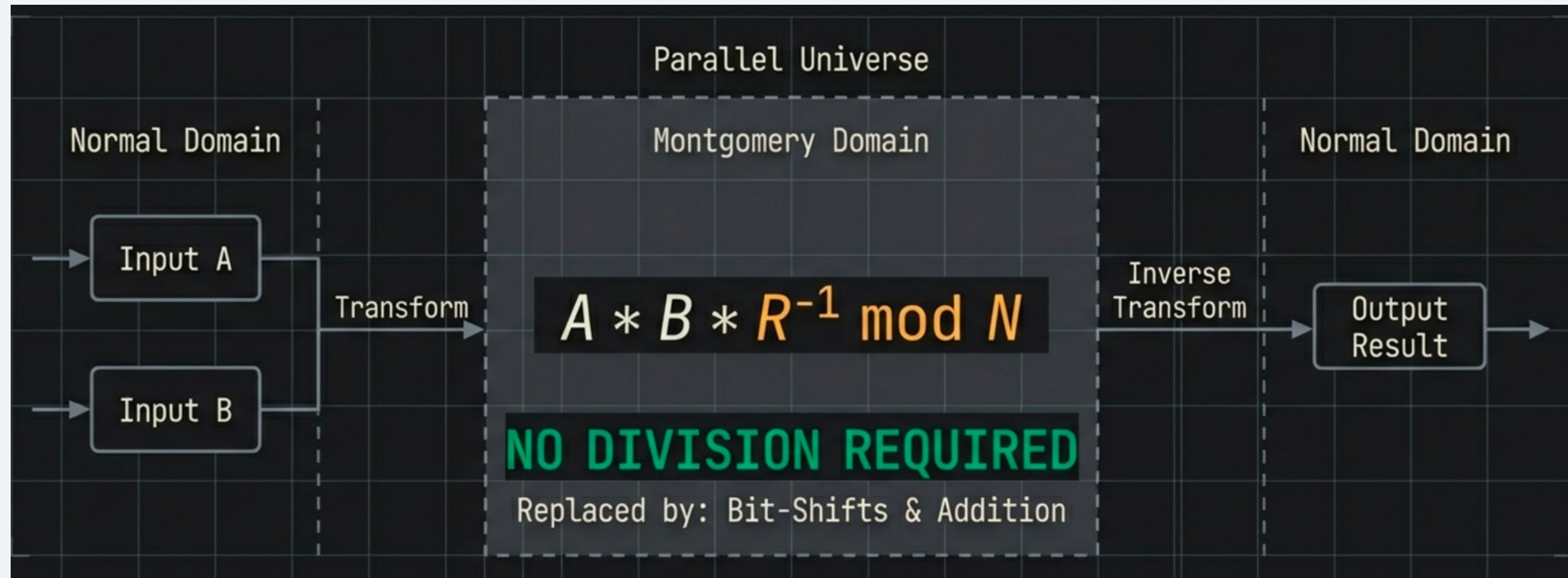


The Engine: Square and Multiply Algorithm

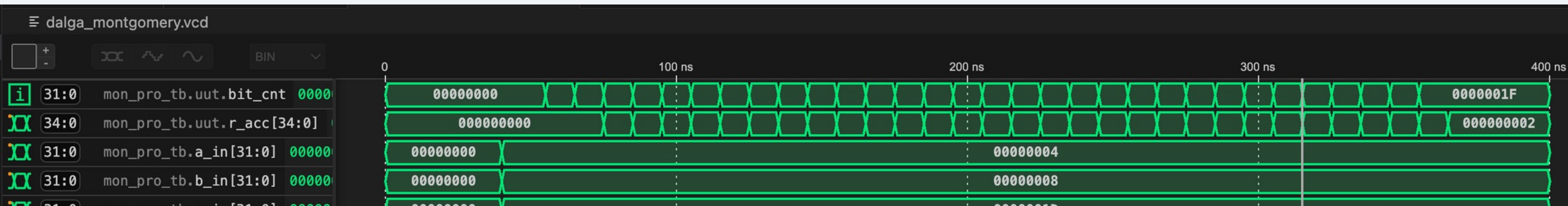
- Logic: Scans the exponent from MSB to LSB.
- Efficiency: Reduces a massive calculation to a series of manageable iterative steps.



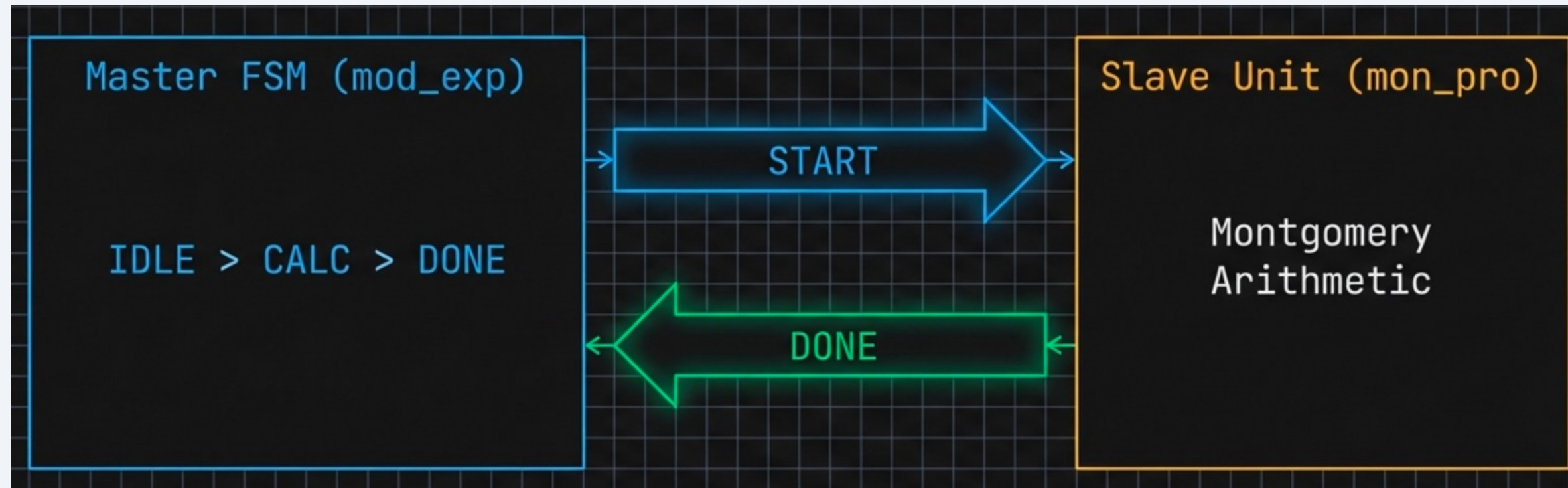
The Optimization: Montgomery Multiplication



- The Concept: Transform operands into a mathematical domain where 'mod n' becomes a simple shift.
- Hardware Advantage: Shifting is free; Division is costly.



System Architecture: Master-Slave FSM



- Master (mod_exp): The Orchestra Conductor. Manages the Square-and-Multiply flow.
- Slave (mon_pro): The Calculator. Performs the specific Montgomery math bit-by-bit.

The Final Trick: Domain Conversion

The Problem

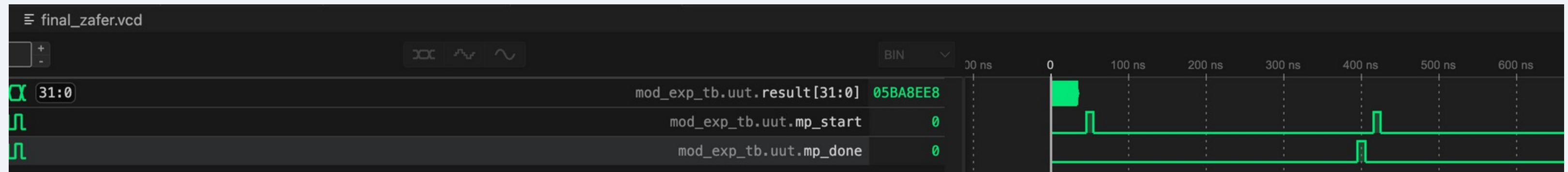
After exponentiation, the result is still
in
Montgomery Form. Raw output
looks like random hex data.

The Fix (Post-Computation)

Trigger the engine one last time with
“1”.

- Impact: Converts result back to human-readable format, matching theoretical Python verification.

RSA Verification & Results



- Status: Road Legal. The module successfully matches standard Python implementations.

Summary of Achievements

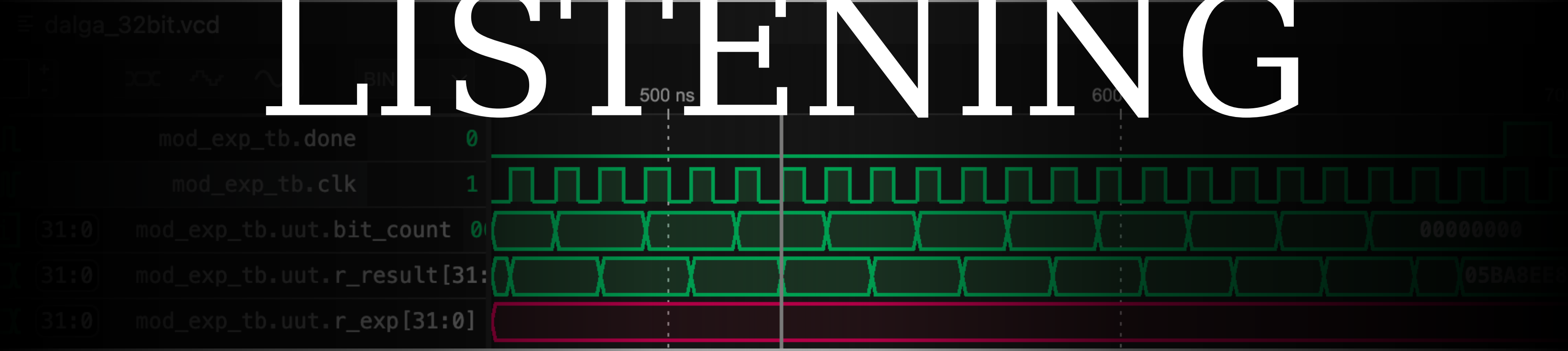
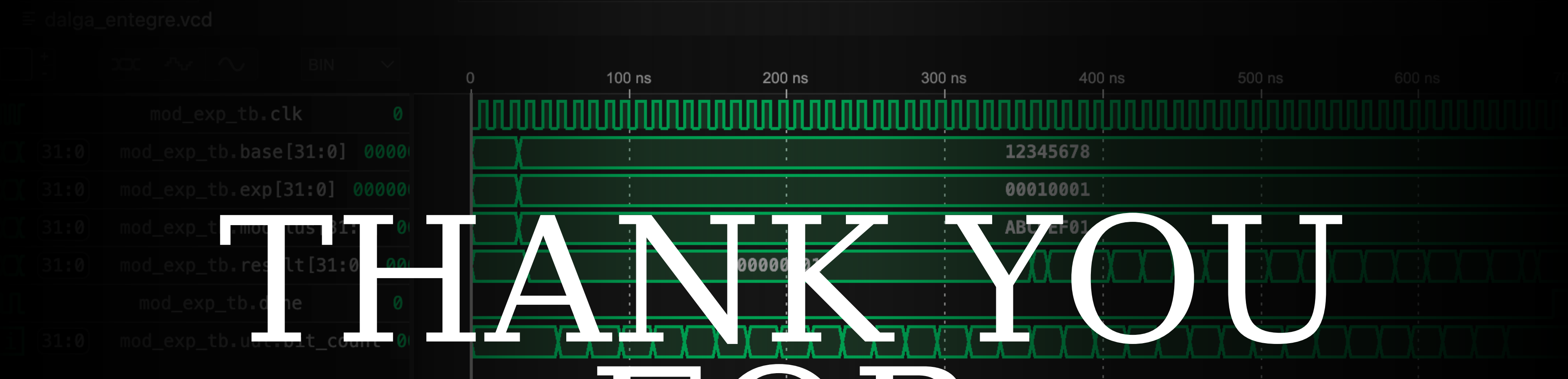
LFSR / Shrinking Generator

- A nonlinear stream encryption method was applied.
- Maximized period ($\sim 2^{127}$) using coprime 64/63-bit primitive polynomials.
- Verified asynchronous "shrinking" behavior.

RSA / Montgomery Engine

- Implemented hardware-optimized Modular Exponentiation
- eliminated costly division using Montgomery arithmetic.
- Designed robust Master-Slave FSM architecture.

THANK YOU
FOR
LISTENING



REFERENCES

